Medical AI Course: Project Guidelines and Rubric

This document details the required deliverables, submission procedures, and grading rubric for Medical AI course projects, including report expectations, GitHub repository standards, and peer evaluations.

Submission Requirements

1. Project Report (PDF)

- 4+ pages, single-spaced, 11-pt Times New Roman
- Filename: Project#_TeamName_Report.pdf
- One submission per team on Canvas

Must include: Abstract, Introduction, Literature Review (≥10 citations), Methods & Data, Results & Evaluation (figures/tables), Discussion & Limitations, References (≥15 total citations).

2. Public GitHub Repository

- Public repo with link in report
- Clear folder structure and meaningful commits

Must include:

- **README.md** (overview, clinical context, setup, run instructions, data info, results summary)
- Well-documented, reproducible code
- **requirements.txt** with package versions

3. Peer Evaluations

- Individual submission (each student)
- Evaluate team members using 1–3 criteria (e.g., contribution, reliability, collaboration)
- Submitted via Canvas

Detailed Deliverable Requirements

1. Research Paper-Style Report (PDF)

Format Requirements:

- **Length**: 4 >= pages, 11-point Times New Roman, single-spaced

- Figures count toward page limit
- **File naming**: [ProjectNumber]_[TeamName]_Report.pdf (e.g., Project1_TeamAlpha_Report.pdf)
- **Submission**: Only one team member needs to submit to Canvas

Required Sections:

Abstract (0.25–0.5 page)

- Concise summary of the problem, method, and key results
- Should be understandable to someone unfamiliar with the project

Introduction (0.75–1 page)

- Clinical/medical background and motivation
- Why this problem matters in healthcare
- Brief description of the intended clinical use case or beneficiary population
- Clear statement of the project goals and objectives

Literature Review (0.75–1 page)

- Review of existing research related to the medical problem or clinical task
- Minimum of **7 citations** for this section
- Discuss relevant ML methods, prior work, and state-of-the-art approaches
- Highlight gap that your project addresses

Methods & Data (1–1.5 pages)

- **Data Description**: Source, size, characteristics, preprocessing steps, train/validation/test split
- **System Architecture**: Overview of the approach (pipeline, model architecture, workflow)
- **Implementation Details**:
    - Specific algorithms or models used
    - Feature engineering or preprocessing decisions
    - Hyperparameter selection process
    - Libraries and tools (scikit-learn, PyTorch, TensorFlow, Hugging Face)
- **Validation Strategy**: Cross-validation, evaluation metrics, statistical tests if applicable

Results & Evaluation (1–1.5 pages)

- **Quantitative Results**: Present performance metrics in tables or figures
  - Include baseline comparisons if applicable
  - Report confidence intervals or standard deviations
- **Qualitative Analysis**: Discuss findings, successful cases, failure cases
- **Visualizations**: Include 2–4 figures with descriptive captions
  - Confusion matrices, ROC curves, sample predictions, model behavior
  - For images projects: sample predictions on test images
  - For LLM projects: example outputs with qualitative analysis
- **Clinical Relevance**: Discuss how results translate to clinical practice

Discussion & Limitations (0.75–1 page)

- Interpretation of results in clinical context
- **Limitations**: Dataset limitations, model assumptions, generalization concerns
- **Ethical Considerations**: Bias, fairness, privacy, clinical validation needs
- **Future Work**: Improvements, extensions, next steps toward clinical deployment

References

- Use consistent citation format (APA, IEEE, or other academic standard)
- Minimum of 15 total citations across report

---

2. GitHub Repository

Requirements:

Repository Setup

- Repository must be **Public**
- Include GitHub link in the report title page
- Team member names and roles clearly documented
- Clean, logical folder structure

README.md Requirements

- **Project Title** and one-sentence summary
- **Clinical Context**: Who this is for and what clinical problem it addresses

- **Quick Start**:
  - Installation instructions (Python version, `pip install -r requirements.txt`)
  - Exact command(s) to run the pipeline
  - Expected runtime and computational requirements
- **Usage Guide**: Step-by-step instructions with expected outputs
- **Data Description** (if applicable):
  - Source(s) of the data
  - Data format and structure
  - License and citations
  - Instructions for obtaining data (if not included)
- **Results Summary**: Key metrics and findings
- **Project Structure**: Brief description of each file/folder
- **Authors and Contributions**: Team member names and roles
- **Dependencies**: Complete list of libraries and versions

Code Quality Standards

- **Documentation**: Docstrings for all functions, inline comments explaining complex logic
- **Reproducibility**: Set random seeds, ensure outputs are reproducible
- **Best Practices**:
  - Meaningful variable and function names
  - PEP 8 compliance (Python code style)
  - No hard-coded paths (use configuration files or arguments)
  - Error handling and logging
- **Commit History**: Meaningful commit messages (e.g., "Add data preprocessing pipeline", "Implement model training", not "fix" or "asdf")

requirements.txt

- List all Python packages and versions used

3. Peer Evaluations

A Canvas assignment will be created for each project that requires every student to complete an individual peer evaluation for their team. Students will assess each teammate (excluding themselves) using the criteria defined below:

- **Preparedness & Engagement:** Comes prepared to meetings, contributes ideas, and stays engaged throughout the project.
- **Contribution** – Consistent, meaningful work toward project goals.

For each assessment, just provide a score (0-10). You are also welcome to provide your reasons for your score.

Grading Rubric

Report (60 points)

Category	Criteria	Points	Notes
Introduction	Clearly articulates clinical motivation and problem statement. Defines the intended use case and target population.	10	Should justify why ML is needed
Related Works	Relevant literature review with appropriate synthesis of prior work.	10	Minimum of **10 peer-reviewed citations**. Should position the project relative to existing approaches.
Methods & Data	Dataset clearly described (source, size, features, labels). Preprocessing steps are justified. Exploratory Data Analysis (EDA) strategies are explained with interpretation of findings. Algorithm selection is well-motivated.	15	Should explain train/test split, feature engineering
Results & Evaluation	Results are clearly presented using appropriate performance metrics. Baseline comparisons are included. Visualizations support interpretation.	10	Expected artifacts include confusion matrix, ROC/PR curves (if applicable), and performance summary tables.
Discussion & Limitations	Provides thoughtful interpretation of results. Discusses limitations, generalizability, and relevance to clinical deployment.	15	Must address ethical considerations (e.g., bias, data leakage, misuse, or patient safety).

GitHub Repository (40 points)

Category	Criteria	Points	Notes / Expectations
Code Organization & Quality	Logical and consistent folder structure. Code is readable,	10	Use meaningful variable/function names,

	modular, well-documented, and reproducible.		docstrings for all functions/classes, and concise inline comments where necessary. Avoid hard-coded paths.
Documentation (README)	Comprehensive README with all required sections and clear instructions.	10	Must include: project overview, problem statement, data source(s), environment setup, **quick start**, usage instructions, and description of repository structure.
Exploratory Data Analysis (EDA)	Dedicated EDA notebook that systematically explores the dataset and informs modeling decisions.	10	Notebook should include: dataset overview, missingness analysis, class/label distributions, feature summaries, key visualizations, and **interpretation of findings**. EDA insights should motivate preprocessing choices and model design.
Demo Notebook	Dedicated demo notebook that allows users to load the trained model, run inference, and evaluate performance.	10	The notebook should include plots or visualizations that help interpret model performance.

Submission Instructions

Canvas Submission

1. **One team member submits all deliverables**:
    - Research paper PDF (named according to format: `Project[N]_[TeamName]_Report.pdf`)
    - GitHub repository link (included in report and in submission comments)
2. **Include in report header**:
    - GitHub repository URL
    - Team member names and roles/contributions

- Submission date

GitHub Submission

1. Ensure repository is **Public**
2. Ensure all code is uploaded and documentation is complete
3. Confirm README has clear instructions for running the project
4. Ensure Git history shows meaningful commits from all team member

Peer Evaluations

1. A Canvas assignment will be created for each project
2. You should list each group member and provide scores for each of the criteria defined above.
3. Evaluations are mandatory (You will not be considered for extra credit at the end of the semester if you did not submit your evaluations from each project)

General Guidelines

Academic Integrity

- Your code, analysis, and report must be your own work
- Cite all sources, including open-source code and libraries
- Use of GitHub Copilot or other AI coding assistants is permitted, but you remain responsible for understanding and testing all code
- Disclose any AI assistance used in your report Methods section

Collaboration

- Each team member should contribute meaningfully
- Clearly document roles and contributions in README

Data Access

- Use publicly available datasets only (not proprietary clinical data)
- Respect data licenses and cite data sources
- Consider data privacy and ethical use in your project design

Code Standards

1. Use the project template repository
2. Comment and document your code thoroughly

3. Follow PEP 8 style guidelines
4. Make code reproducible and portable